NETWORK DATA ANALYSIS LABORATORY · TEAM S

Throughput Prediction in a Dense 5G Deployment

Operator measurement devices as network-state features · Random Forest vs. MLP · Federated Learning with FedAvg

Pablo Garcia · Esteban Lopez · Pietro Limoni

Project #7

Agenda

01 **The problem & requirements**

What we're predicting, and how the assignment maps to our experiments

02 **Dataset & EDA**

ACC Arena subset, target distribution, traffic-type coverage

03 **Feature engineering**

Operator measurement devices — the Team 7-specific idea

04 **Methodology**

Chronological split, time-aware CV, model choices

05 **Centralized models**

Random Forest vs. MLP, tuning, results

06 **Impact of X**

Sensitivity of results to the number of operator devices

07 **Federated Learning**

FedAvg vs. centralized and local-only baselines

08 **Discussion**

Conclusions, limitations, and what we'd do next

CONTEXT

The problem, in one sentence

Can we predict a user's throughput from network and radio measurements alone without directly measuring it in a Dense 5G Network?

- Directly probing every device for throughput is expensive and doesn't scale to a dense venue with thousands of users.
- But the network already knows signal quality (SINR), resource allocation (PRB), error rates (BLER) and what the user is doing (traffic type).
- If those signals predict throughput well, operators can monitor network health, plan capacity, or flag congestion without instrumenting every device.

12,000

users in the ACC Arena venue (full dataset)

6 classes

traffic types: off, idle, constant-rate, video, gaming, http

CONTEXT

Mapping every requirement to an experiment

1	**Throughput regression on ACC Arena users**	Dataset filtering + target = throughput_mbps
2	**Compare Random Forest and a Neural Network**	Centralized RF vs. tuned MLP
3	**Use BLER, PRB, SINR, traffic type, + operator devices**	Base features + operator_same_type_* features
4	**Test different values of X**	X-sweep: X ∈ {3, 5, 10, 20}
5	**Advanced task: Federated Learning**	FedAvg over per-user clients vs. centralized / local baselines

Three goals structure the analysis

GOAL 1

Centralized prediction

Random Forest vs. MLP on the full feature set, tuned with cross-validation.

GOAL 2

Impact of X

How the number of operator measurement devices changes prediction quality.

GOAL 3

Federated Learning

FedAvg vs. local-only and centralized baselines, simulating per-user privacy.

DATASET

ACC Arena dataset overview

Scenario

- Dense 5G deployment simulation in the ACC Arena venue (Liverpool).
- Each row = one user, at one timestamp, described by radio, traffic, association and position information.
- Prediction target: **throughput_mbps**, the estimated user's throughput from terminal and radio-unit features.

Main raw columns

Identifiers	`venue, timestamp, user_id`
Traffic / association	`traffic_type, ru_id`
Radio metrics	`sinr_dl, sinr_ul, prb, bler`
Position	`x, y, z`
Target	`throughput_mbps`

DATASET

Extracted subset and reproducibility

108,240
rows in the ACC Arena subset

220
users, IDs 0–219

492
samples per user (balanced)

98,400
final prediction rows at X = 20

Why a subset?

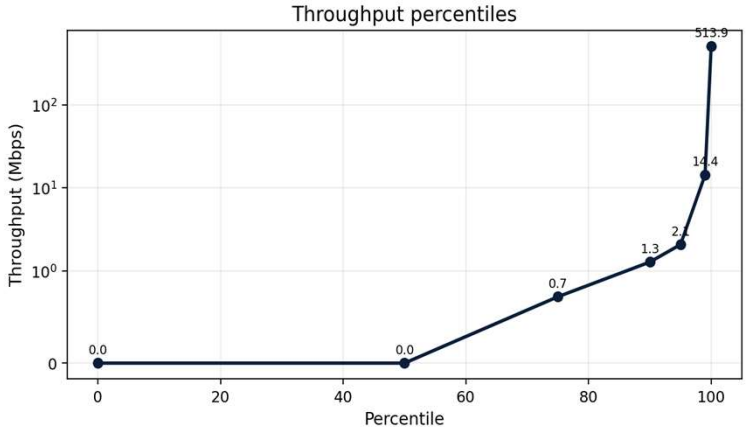
- The full dataset (12k users × 10k samples) is too large to iterate on in Google Colab.
- The subset keeps an equal, balanced number of samples per user while still allowing the full pipeline — X-sweep and FedAvg included — to run end to end.

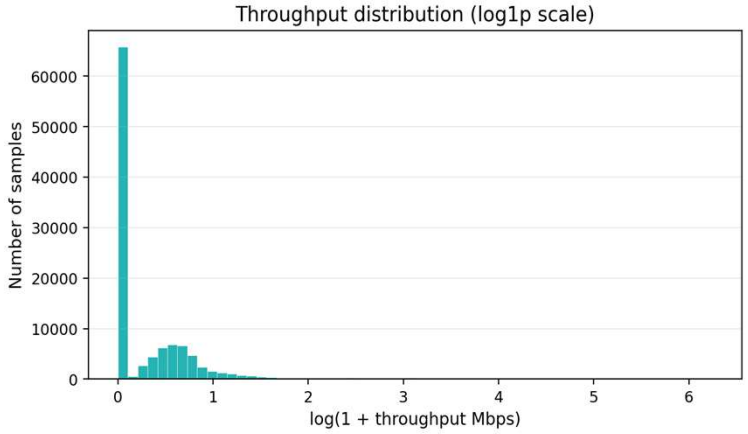
Extraction configuration

```
measurement_device_count = 20
target_user_start = 20
target_user_count = 200
time_stride = 20
test_fraction = 20% (chronological)
```

EDA

Target distribution: throughput is highly skewed





0.787 Mbps
Mean

14.368 Mbps
99th pct.

0.000 Mbps
Median

513.936 Mbps
Maximum

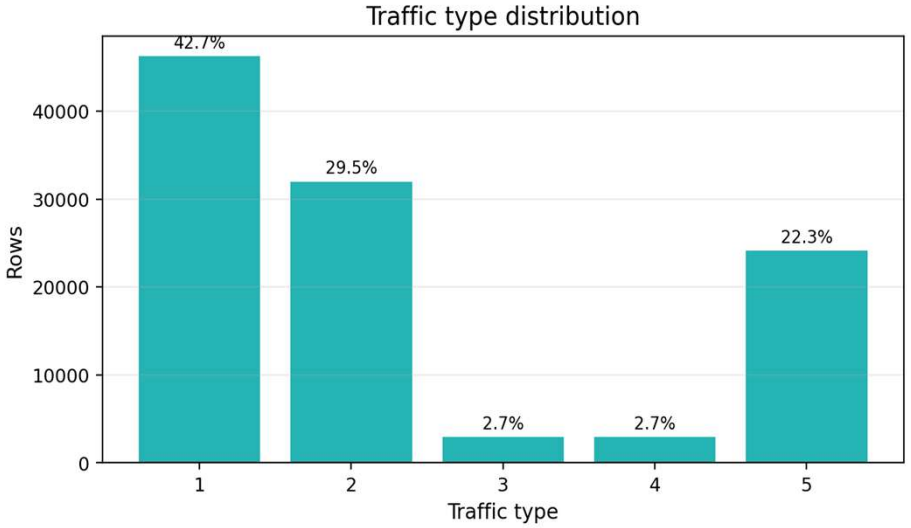
2.080 Mbps
95th pct.

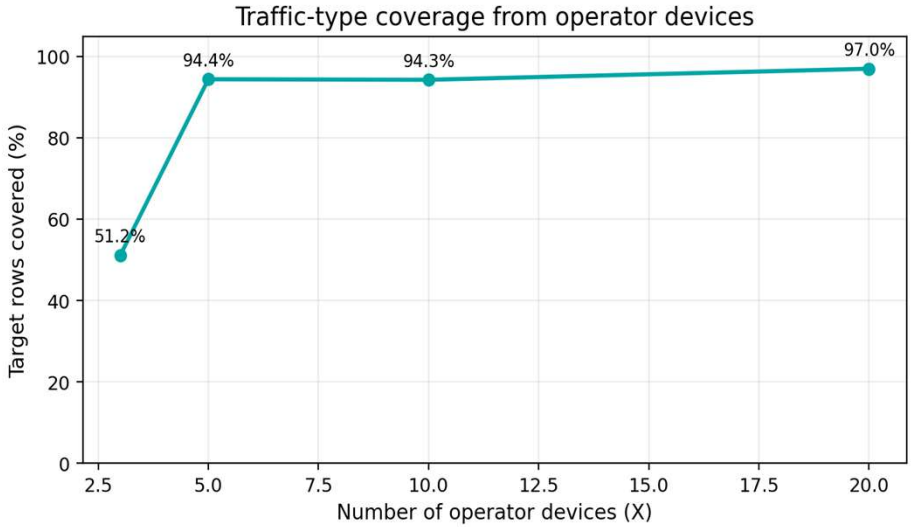
60.8%
Zero share

60.8% of samples have exactly zero throughput, while a rare few spike above 100 Mbps.

EDA

Traffic types and operator-device coverage





Insights

Operator features are computed only from operator devices sharing the target user's traffic type. Rare traffic types (3, 4 — 2.7% each) need a larger X to be reliably represented, this is what the X-sweep is trading-off.

Operator measurement devices

Two groups of users

Operator measurement devices

Users 0 to X−1. Act like network probes — their throughput can be used as a feature for other users. They are excluded from the prediction set entirely.

Target users

All remaining users. Their own throughput is never used as an input — only predicted. This is what prevents label leakage.

The idea being

If probe devices watching video are averaging 2 Mbps right now, that's a live signal for how congested the network currently is for video traffic which then should correlate with what any other video-watching user nearby is getting at that same moment.

Worked example: how one feature value is built

1	**Pick a target row**	`User 45, traffic_type = video, timestamp = t = 1215s`
2	**Find matching probes**	`All operator devices (IDs 0–19) with traffic_type = video, at t = 1215s`
3	**Aggregate their throughput**	`count, mean, median, max, min, std of those probes' throughput_mbps`
4	**Attach to the target row**	`These 6 numbers become user 45's operator_same_type_* features at t = 1215s`

If no operator device shares that traffic type at that timestamp, the six columns are filled with 0.0 since there’s no signal available.

Operator-device feature engineering

`operator_same_type_count`	Number of operator devices sharing the target's traffic type
`operator_same_type_mean_throughput`	Average observed throughput for that traffic type
`operator_same_type_median_throughput`	Median observed throughput
`operator_same_type_max / min_throughput`	Range of same-type operator throughput
`operator_same_type_std_throughput`	Variability of same-type operator throughput

Computation rule:

Group operator devices by (timestamp, traffic_type), aggregate throughput statistics, then left-merge those statistics onto target-user rows. Operator devices are dropped from the prediction frame afterward.

Final feature set

Original features

- sinr_dl, sinr_ul (signal quality)
- prb (allocated resource blocks)
- bler (block error rate)
- x, y, z (position)
- traffic_type, ru_id (categorical → one-hot)

Engineered operator features

- operator_same_type_count
- ..._mean_throughput
- ..._median_throughput
- ..._max / min_throughput
- ..._std_throughput

Target variable: throughput_mbps

From raw CSV to model-ready table

Encoding & scaling

- traffic_type and ru_id are categorical → one-hot encoded.
- Numeric features (SINR, PRB, BLER, position, operator stats) are standardized (mean 0, std 1) — mainly for the MLP.

Why a pipeline object?

- Preprocessing and model are wrapped in one sklearn Pipeline / ColumnTransformer.
- This avoids fitting the scaler on the full dataset — a common leakage mistake.

Why a chronological, per-user split?

A random split would let the model train on second 500 of a user and be tested on second 480 of the same user, peeking slightly into the future to predict the past.

The rule, per user

```
for user_id, user_frame in frame.groupby(user_id):
    user_frame = user_frame.sort_values(t)
    split = int(len(user_frame) * 0.8)
    train += user_frame[:split]   # oldest 80%
    test  += user_frame[split:]   # newest 20%
```

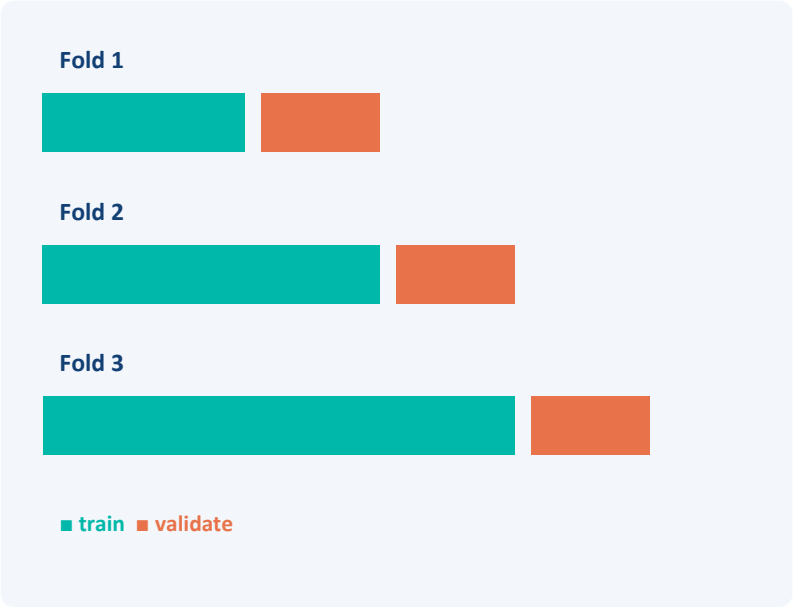
What this guarantees

- 80% oldest samples per user → training.
- 20% newest samples per user → test.
- The test set is always chronologically after the training data it corresponds to — realistic for a deployment where you only ever have the past to predict the near future.

METHODOLOGY

Time-aware cross-validation for tuning

- Standard k-fold CV shuffles rows into random folds — wrong here for the same reason a random split is wrong.
- TimeSeriesSplit instead trains on an earlier block of time and validates on a later block, repeated across folds, the rows handed to it are sorted by time.
- Used inside GridSearchCV for both Random Forest and MLP hyperparameter search.



METHODOLOGY

Hyperparameter tuning grids

Random Forest

```
n_estimators: [200, 300]
max_depth: [12, 16, None]
min_samples_leaf: [1, 2, 4]
```

54 fits · 3-fold TimeSeriesSplit

MLP (Neural Network)

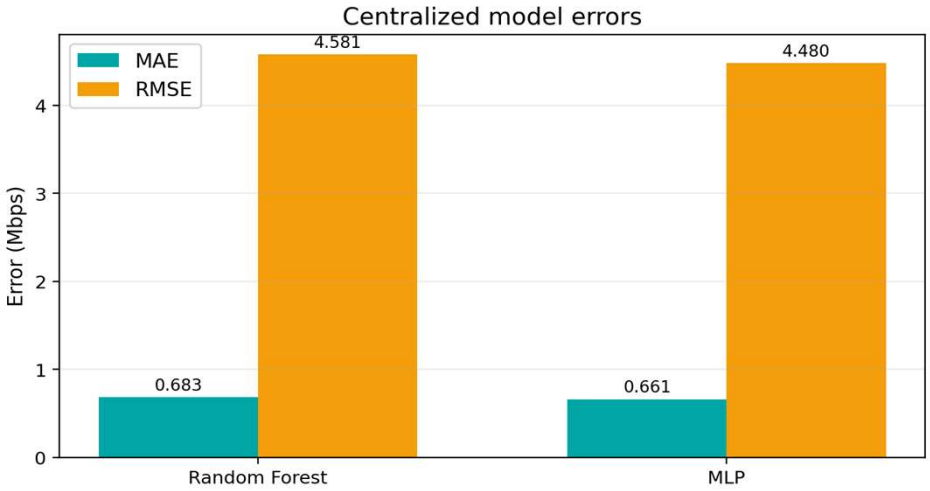
```
hidden_layer_sizes: [(128,64), (256,128)]
alpha: [1e-4, 1e-3]
learning_rate_init: [1e-3, 5e-4]
```

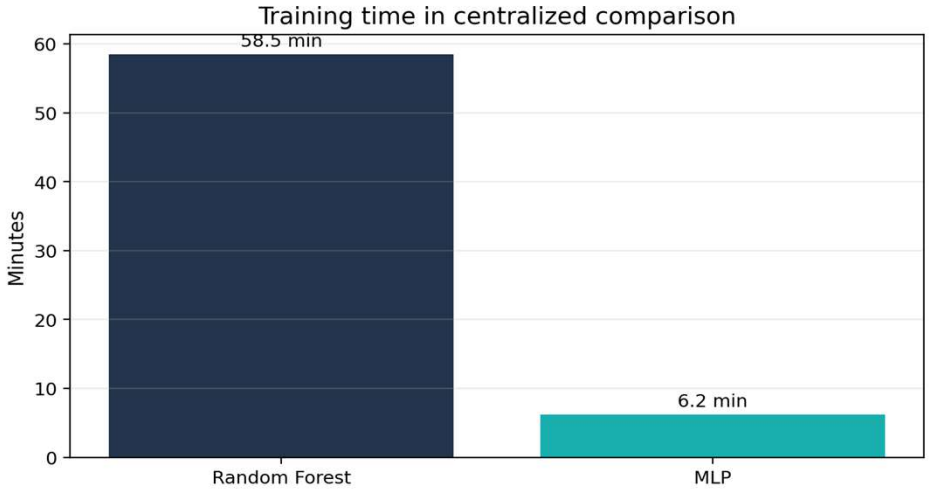
24 fits · 3-fold TimeSeriesSplit

Metrics tracked for every fit: MAE, MSE, RMSE, R^2, and training time.

Centralized results (X = 20)

Model	MAE	RMSE	R^2	Training time
Random Forest	0.683	4.581	0.015	58.5 min
MLP	0.661	4.480	0.058	6.2 min





Interpretation: MLP is the better trade-off

MLP

best centralized model - lower MAE/RMSE, higher R^2

58.5 min

Random Forest training cost - GridSearchCV is expensive

6.2 min

MLP training cost - far better cost/performance

The centralized MLP is the best trade-off in this experiment. The improvement over Random Forest is moderate but consistent, and the computational saving is large.

A caution on training time

Training time includes hyperparameter tuning and was measured in Google Colab, It is treated as an indicative computational-cost signal.

Why is R^2 so low?

Target distribution

- 60.8% of samples have zero throughput.
- Median throughput = 0 Mbps.
- Only a few samples reach very high values.

Metric behavior

- MAE is dominated by common low-throughput cases.
- RMSE and R^2 are strongly affected by rare high-throughput peaks (errors are squared).

Interpretation

- Models reduce average error well.
- But they don't fully explain the variance of throughput in a dense 5G scenario.

This low R^2 here is a property of the target distribution, not a modeling mistake.

X-sweep setup

X ∈ {3,5,10,20} → **Rebuild dataset (features change)** → **Train RF + MLP (same pipeline)** → **Same evaluation logic**

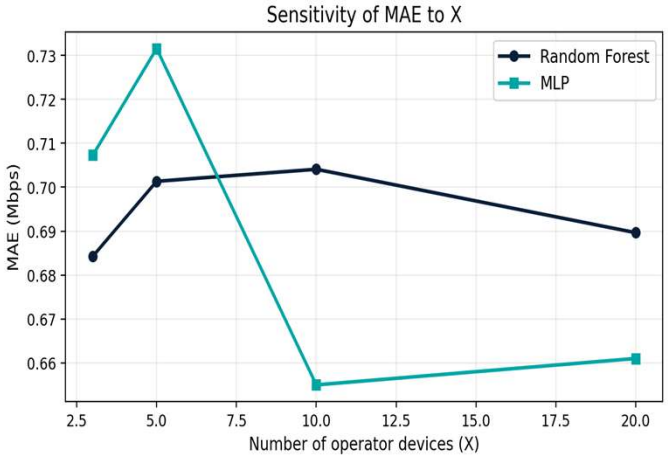
Why X may help

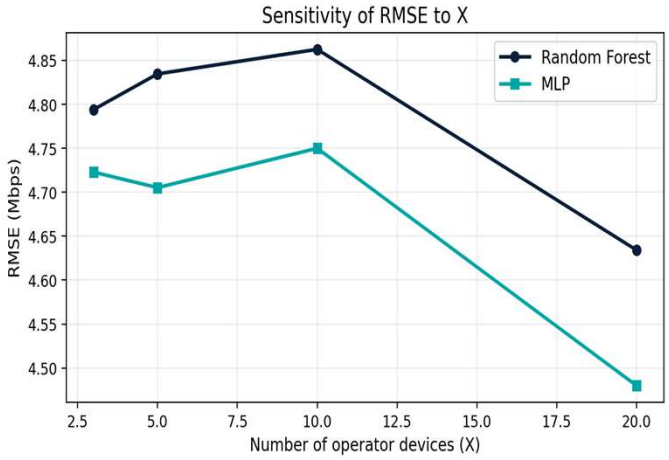
More operator devices provide richer, more stable same-traffic information about the current network state.

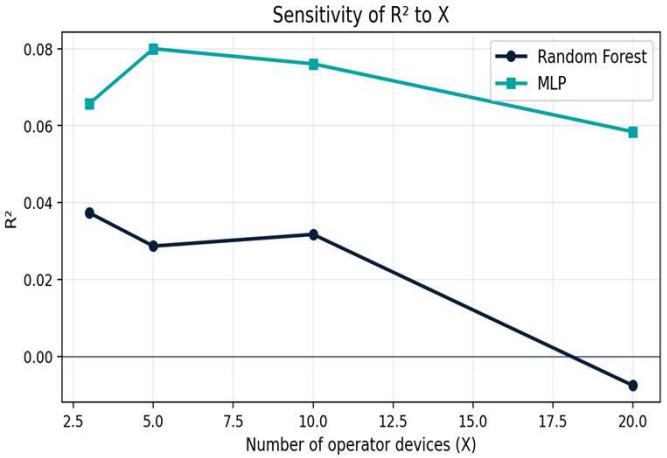
Why X may not always help

A larger X excludes more users from the prediction dataset and changes which traffic types are covered by probes.

X-sweep results







Metric	Best RF	Best MLP
MAE	X = 3	X = 10
RMSE	X = 20	X = 20
R^2	X = 3	X = 5

X-sweep interpretation

Increasing X does not consistently improve all metrics. The best X depends on the model and the metric.

Reason 1

More operator devices provide more network context and better traffic-type coverage.

Reason 2

A larger X excludes more users from the prediction set, so the target dataset itself changes across experiments.

Operator measurement features are useful, but “more operator devices” is not automatically “better prediction.”

What does FedAvg do?

- Centralized training pools every user's data on one machine. FedAvg instead keeps each user's data on their own device — only model updates are shared, never raw data.
- Motivation: realistic for a telecommunications operator that can't centralize subscriber-level telemetry for privacy reasons.
- Question being tested: does a federated global model do as well as (a) a fully centralized model, and (b) a model trained purely locally per user with no collaboration at all?

Client

Each user = one federated client, training only on their own rows.

Local training

3 local epochs per round, before sending weights back.

Server aggregation

FedAvg — weighted average of client weights by sample count.

FedAvg, step by step

1	**Broadcast**	Server sends the current global model to a batch of clients (users).
2	**Local training**	Each client trains locally for a few epochs on only their own data.
3	**Upload**	Each client sends back their updated model weights (not their raw data).
4	**Weighted averaging**	Server averages weights, weighted by each client's number of samples.
5	**Repeat**	Steps 1–4 repeat for 20 federated rounds.

```
averaged[key] = Σ ( state[key] × weight_i / total_weight )   — weighted by client sample count
```

Experiment configuration

= users	20	3	50
Clients	Federated rounds	Local epochs	Min. samples / client

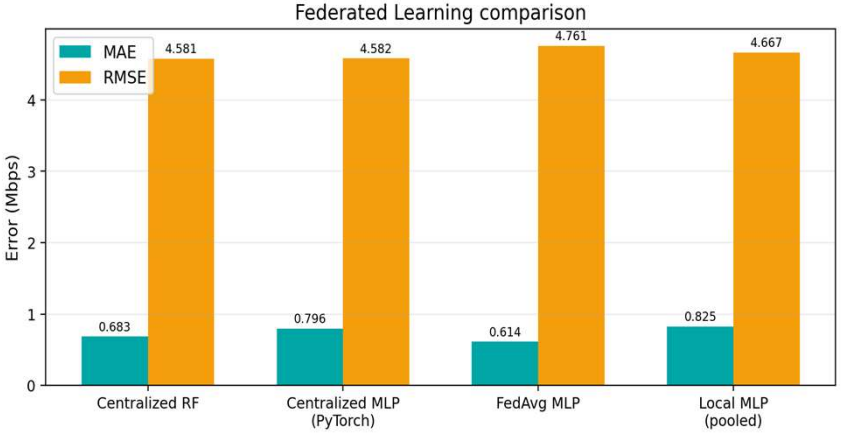
Model architecture (PyTorch)

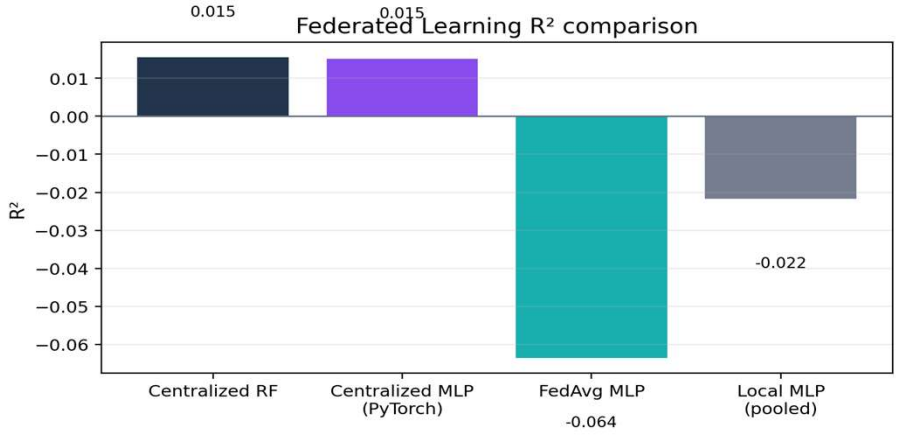
```
Input  →  128  →  64  →  Output

Activation: ReLU
Loss: MSE
Optimizer: Adam
```

Baselines compared

- Centralized Random Forest
- Centralized PyTorch MLP (untuned — architecturally fixed to match the FL client model)
- FedAvg global MLP
- Local-only MLP, pooled evaluation across all per-user models

Federated Learning results





Model	MAE	RMSE	R^2
Centralized RF	0.683	4.581	0.015
Centralized MLP (PyTorch)	0.796	4.582	0.015
FedAvg MLP	0.614	4.761	-0.064
Local MLP (pooled)	0.825	4.667	-0.022

Why does FedAvg beat MAE but lose on R^2?

- FedAvg gets the best MAE of all four models (0.614), it's good at the common, low-throughput case.
- But it has the worst R^2 (−0.064), it’s worse than always predicting the mean.

Likely mechanism: traffic-type heterogeneity across clients

Each user's local model trains almost exclusively on that user's dominant traffic type. Averaging together ~200 such specialized local models with FedAvg smooths out client-specific patterns including the rare high-throughput cases a single centralized model can capture by seeing all traffic types jointly. That washing-out effect hurts RMSE/R^2 (sensitive to rare large errors) more than MAE (dominated by the common near-zero case).

Conclusion and Discussion

- MLP is the best centralized model in this experiment.
- Random Forest is slightly worse and much slower to tune.
- Operator-device features provide useful context, but the effect of X is not monotonic.
- FedAvg improves MAE but not RMSE or R^2 — traffic-type heterogeneity likely dilutes rare high-throughput patterns.
- Throughput prediction remains challenging in dense 5G scenarios due to the heavily skewed target.

Limitations

- Compact subset used instead of the full 12k-user dataset.
- Highly skewed target distribution (60.8% zeros).
- Only 4 values of X tested.
- Two different MLP implementations (sklearn vs. PyTorch) are not directly comparable.

Improvements

- Log-transform the target (log1p) to reduce skew sensitivity.
- Run on the full dataset and a wider X range.
- Improve traffic-type coverage of operator probes at small X.
- Tune the PyTorch MLP the same way the centralized models were tuned.

BACKUP SLIDES

Use only if asked for extra detail or code inspection

BACKUP

Where to find it in the notebook

How was the dataset loaded?	Sections 2 and 4
How are operator features built?	Section 6 — preprocessing & feature engineering
How is the test split created?	Section 7 — train/test split
Where are RF and MLP trained?	Section 8 — centralized models
Where is FedAvg implemented?	Section 9 — Federated Learning
Where is X tested?	Section 10 — experiment with different X values

Key code: operator features & FedAvg aggregation

build_operator_features / prepare_dataset

```
def build_operator_features(frame, X):
  op = frame[frame.user_id.between(0, X-1)]
  return (op.groupby([t, traffic_type])
            [throughput]
            .agg(['count','mean','median',
                  'max','min','std'])
            .reset_index())

def prepare_dataset(raw, X):
  feats = build_operator_features(raw, X)
  pred = raw[~raw.user_id.between(0, X-1)]
  pred = pred.merge(feats, on=[t, traffic_type],
                    how='left').fillna(0.0)
```

average_state_dicts (FedAvg)

```
def average_state_dicts(state_dicts, weights):
  total = float(sum(weights))
  out = OrderedDict()
  for key in state_dicts[0]:
    out[key] = sum(
      s[key] * (w / total)
      for s, w in zip(state_dicts, weights)
    )
  return out

# weighted by each client's sample count
```

BACKUP

Interpreting apparent discrepancies

Two centralized MLPs

Main comparison uses a tuned scikit-learn MLP. The FL section uses an untuned PyTorch MLP baseline (needed to share weight-averaging code with FedAvg clients). They should not be treated as identical models.

RF main vs. RF at X=20

The X-sweep uses a reduced hyperparameter grid for speed, so its X=20 result can differ slightly from the main centralized RF result (R^2 0.015 vs. −0.008).

Local-avg vs. pooled evaluation

Averaging RMSE per client is not the same as sqrt(global MSE). Local MLP (pooled) is the fairer baseline for comparison with FedAvg and centralized models.

METHODOLOGY

Two model families, required by the assignment

Random Forest

- Ensemble of many decision trees, each on a random subset of rows/features.
- Predictions are averaged across trees.
- Handles nonlinear interactions natively, robust to unscaled features.
- Strong default baseline for tabular data.

MLP (Neural Network)

- Layers of neurons with nonlinear activations, trained by backpropagation.
- Requires standardized inputs to train well.
- The Neural Network required by the assignment.
- Two implementations used: a tuned scikit-learn MLP here, a PyTorch MLP later for Federated Learning.